

CUESTIONARIO 4: Core APIs

1. What is output by the following code? (Choose all that apply.)

```
1: public class Fish {  
2:     public static void main(String[] args) {  
3:         int numFish = 4;  
4:         String fishType = "tuna";  
5:         String anotherFish = numFish + 1;  
6:         System.out.println(anotherFish + " " + fishType);  
7:         System.out.println(numFish + " " + 1);  
8:     } }
```

- A. 4 1
- B. 5
- C. 5 tuna
- D. 5tuna
- E. 51tuna
- F. The code does not compile.

Mi justificación

La línea 6 imprime "5 tuna" ya que la línea 5 ejecuta la operación y luego la convierte a String. Pero numFish no cambia a 5 porque solo lo hace para el String.

La línea 7 imprime este valor de numFish que no cambio, y como viene un " ", entonces ya lo que sigue se maneja como String, ósea ya no hay operaciones aritméticas. Así que imprime 4 1.

1. F. La línea 5 no compila. Esta pregunta busca comprobar si estás prestando atención a los tipos. numFish es un int, y 1 es un int. Por lo tanto, usamos suma numérica y obtenemos 5. El problema es que no podemos almacenar un int en una variable String. Supongamos que la línea 5 dijera `String anotherFish = numFish + 1 + ""`; En ese caso, las respuestas serían la opción A y la opción C. La variable definida en la línea 5 sería el String "5", y ambas instrucciones de salida usarían concatenación.

2. Which of these array declarations are not legal? (Choose all that apply.)
- A. `int[][] scores = new int[5][];`
 - B. `Object[][][] cubbies = new Object[3][0][5];`
 - C. `String beans[] = new beans[6];`
 - D. `java.util.Date[] dates[] = new java.util.Date[2][];`
 - E. `int[][] types = new int[];`
 - F. `int[][] java = new int[][];`

Mi justificación
C no es válido pues el lado izquierdo se menciona un arreglo compuesto de Strings, del lado derecho se quiere crear un arreglo de objetos de clase beans. Estos dos tipos String y beans no son el mismo o E no es válido porque el lado izquierdo menciona que se quiere crear un arreglo bidimensional, pero el derecho crea un arreglo unidimensional. Esto no es compatible. Los demás tienen sintaxis válida.

2. **C, E, F.** La opción C usa el nombre de una variable como si fuera un tipo, lo cual claramente es ilegal. Las opciones E y F no especifican ningún tamaño. Aunque es válido omitir el tamaño para las dimensiones posteriores de un arreglo multidimensional, la primera dimensión es obligatoria. La opción A declara un arreglo 2D válido. La opción B declara un arreglo 3D válido. La opción D declara un arreglo 2D válido. Recuerda que es normal encontrar clases en el examen que quizá no hayas aprendido. No se espera que sepas nada sobre ellas.

3. Note that March 13, 2022 is the weekend when we spring forward, and November 6, 2022 is when we fall back for daylight saving time. Which of the following can fill in the blank without the code throwing an exception? (Choose all that apply.)

```
var zone = ZoneId.of("US/Eastern");  
var date = _____;  
var time = LocalDateTime.of(2, 15);  
var z = ZonedDateTime.of(date, time, zone);
```

- A. `LocalDate.of(2022, 3, 13)`
- B. `LocalDate.of(2022, 3, 40)`
- C. `LocalDate.of(2022, 11, 6)`
- D. `LocalDate.of(2022, 11, 7)`
- E. `LocalDate.of(2023, 2, 29)`
- F. `LocalDate.of(2022, MonthEnum.MARCH, 13);`

Mi justificación
B es inválido porque el día 40 no existe para ningún mes. E no es válido porque el día 29 de febrero solo existe para años bisiestos, y 2023 no es año bisiesto.

3. A, C, D. El código compila correctamente. La opción B lanza una excepción porque no existe el 40 de marzo. La opción E también lanza una excepción porque 2023 no es un año bisiesto y, por lo tanto, no existe el 29 de febrero. La opción F no compila porque el enum debe llamarse Month, no MonthEnum. La opción D es correcta porque simplemente es una fecha normal y no tiene relación con el horario de verano. Las opciones A y C son correctas porque Java es lo suficientemente inteligente como para ajustar el horario de verano.

4. Which of the following are output by this code? (Choose all that apply.)

```
3: var s = "Hello";
4: var t = new String(s);
5: if ("Hello".equals(s)) System.out.println("one");
6: if (t == s) System.out.println("two");
7: if (t.intern() == s) System.out.println("three");
8: if ("Hello" == s) System.out.println("four");
9: if ("Hello".intern() == t) System.out.println("five");
```

- A. one
- B. two
- C. three
- D. four
- E. five
- F. The code does not compile.
- G. None of the above

Mi justificación
A es válida porque equals evalúa el contenido del String, así que ambas contienen "Hello". B es inválida porque t y s son Strings que apuntan a diferentes valores "Hello". C es válida porque el método intern() evalúa el contenido del String, así que ambas contienen "Hello". D es válida porque evaluar "Hello" en comparación significa que evaluarás el valor del String Pool, así que ambos tienen "Hello" en su valor, entonces la condición if da true. E es inválido porque t crea un new String, así que si evaluamos el intern de "Hello" que es el mismo "Hello", dará false ya que son diferentes valores de referencia.

4. **A, C, D.** La opción A apunta al String en el *string pool*. La línea 4 llama explícitamente al constructor de String y, por lo tanto, crea un objeto diferente de s. La línea 5 comprueba la igualdad de objetos mediante .equals(), lo cual es verdadero, por lo que imprime one. La línea 6 utiliza igualdad de referencias (==), lo cual no es verdadero porque tenemos objetos diferentes. La línea 7 llama a intern(), que devuelve la referencia al valor existente en el *string pool* y, por lo tanto, es la misma referencia que s. La línea 8 también compara referencias, pero es verdadera porque ambas referencias apuntan al objeto del *string pool*. Finalmente, la línea 9 es una trampa. El String Hello ya está en el *string pool*, por lo que llamar a intern() no cambia nada. La referencia t apunta a un objeto diferente, por lo que el resultado sigue siendo falso.

5. What is the result of the following code?

```
7: var sb = new StringBuilder();  
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");  
9: System.out.println(sb);
```

- A. abbaaccc
- B. abbaccca
- C. bbaaaccc
- D. bbaaccca
- E. An empty line
- F. The code does not compile.

Mi justificación
El primer append crea "aaa". El primer insert inserta "bb" en la posición 1, así que queda "abbaa". El segundo insert inserta "ccc" en la posición 4 de "abbaa", entonces queda "abbaccca"

5. B. Este ejemplo utiliza *method chaining* (encadenamiento de métodos). Después de llamar a `append()`, `sb` contiene "aaa". Ese resultado se pasa a la primera llamada a `insert()`, que inserta en el índice 1. En este punto, `sb` contiene `abbaa`. Ese resultado se pasa al `insert()` final, que inserta en el índice 4, dando como resultado `abbaccca`.

6. How many of these lines contain a compiler error? (Choose all that apply.)

```
23: double one = Math.pow(1, 2);
```

```
24: int two = Math.round(1.0);
```

```
25: float three = Math.random();
```

```
26: var doubles = new double[] {one, two, three};
```

A. 0

B. 1

C. 2

D. 3

E. 4

Mi justificación
La línea 23 no tiene error. La línea 24 no tiene error porque round regresa un entero. La línea 25 sí tiene un error ya que random() regresa un double, y si se quiere guardar como float se requiere hacer un casteo explícito (float), el cual no tiene. La línea 26 no tiene error porque una variable var no necesita corchetes [] para referenciar un arreglo.

6. C. Recuerda prestar atención a los tipos de retorno de las operaciones matemáticas. Uno de los trucos está en la línea 24. El método round() devuelve un int cuando se llama con un float. Sin embargo, lo estamos llamando con un double, por lo que devuelve un long. El otro truco está en la línea 25. El método random() devuelve un double. Como dos líneas tienen un error de compilación, la opción C es la respuesta.

7. Which of these statements is true of the two values? (Choose all that apply.)

2022-08-28T05:00 GMT-04:00

2022-08-28T09:00 GMT-06:00

- A. The first date/time is earlier.
- B. The second date/time is earlier.
- C. Both date/times are the same.
- D. The date/times are two hours apart.
- E. The date/times are six hours apart.
- F. The date/times are 10 hours apart.

Mi justificación
Convirtiendo a UTC, tenemos que la primera hora es -4 horas a la de UTC, así que en formato UTC es 9:00. Si convertimos la segunda que es -6 horas, obtenemos en formato UTC las 15:00, la diferencia entre estas 2 fechas es de 6 horas. También la A es verdadera ya que UTC 9:00 es menor que UTC 15:00 (en el mismo día claro).

7. A, E. Al trabajar con zonas horarias, es mejor convertir primero a GMT restando la zona horaria. Recuerda que restar un número negativo es como sumar. La primera fecha/hora es 9:00 GMT y la segunda es 15:00 GMT. Por lo tanto, la primera es seis horas anterior.

8. Which of the following return 5 when run independently? (Choose all that apply.)

```
var string = "12345";
```

```
var builder = new StringBuilder("12345");
```

- A. `builder.charAt(4)`
- B. `builder.replace(2, 4, "6").charAt(3)`
- C. `builder.replace(2, 5, "6").charAt(2)`
- D. `string.charAt(5)`
- E. `string.length`
- F. `string.replace("123", "1").charAt(2)`
- G. None of the above

Mi justificación
✓ A es válida porque el carácter en el índice 4 es el 5. ✓ B es válida porque, después de replace tenemos 1265, el carácter en el índice 3 es el 5. ✗ C es inválida porque, después de replace tenemos 126, el carácter en el índice 2 es el 6. ✗ D es inválida porque el último índice posible de builder es 4. ✗ E es inválida porque está mal escrito. Como es un método, deber ser <code>string.length()</code>. ✓ F es válida porque, después de replace tenemos 145, el carácter en el índice 2 es el 5. ✗ G es inválida porque A,B,C y E son válidas.

8. **A, B, F.** Recuerda que los índices comienzan en cero, lo que significa que el índice 4 corresponde al quinto elemento, y la opción A es correcta. Para la opción B, el método `replace()` comienza el reemplazo en el índice 2 y termina antes del índice 4. Esto significa que se reemplazan dos caracteres, y después se llama a `charAt(3)` sobre el valor intermedio 1265. El carácter en el índice 3 es 5, por lo que la opción B es correcta. La opción C es similar: el valor intermedio es 126 y devuelve 6. La opción D produce una excepción porque no existe un carácter en el índice 5. La opción E es incorrecta. No compila porque faltan los paréntesis del método `length()`. Finalmente, el `replace` de la opción F produce el valor intermedio 145. El carácter en el índice 2 es 5, por lo que la opción F es correcta.

9. Which of the following are true about arrays? (Choose all that apply.)
- A. The first element is index 0.
 - B. The first element is index 1.
 - C. Arrays are fixed size.
 - D. Arrays are immutable.
 - E. Calling `equals()` on two different arrays containing the same primitive values always returns `true`.
 - F. Calling `equals()` on two different arrays containing the same primitive values always returns `false`.
 - G. Calling `equals()` on two different arrays containing the same primitive values can return `true` or `false`.

Mi justificación
✓ A es true por definición ✗ B es false porque el primer elemento tiene índice 0. ✓ C es true por definición ✗ D es false por definición. ✓ E es true porque <code>equals()</code> compara el contenido, y si son primitivos no pueden apuntar a diferentes instancias, por lo que siempre darán true dos arreglos con el mismo contenido. ✗ F es false porque la E es true. ✗ G es false porque en E ya se aclaró que es imposible que de false.

9. **A, C, F.** Los arreglos utilizan índices que comienzan en cero, por lo que la opción A es correcta y la B es incorrecta. Los arreglos no pueden cambiar de tamaño, por lo que la opción C es correcta. Los valores sí pueden cambiar, haciendo que la opción D sea incorrecta. Un arreglo no sobrescribe `equals()`, por lo que utiliza la igualdad de objetos. Como dos objetos diferentes no son iguales, la opción F es correcta y las opciones E y G son incorrectas.

10. How many of these lines contain a compiler error? (Choose all that apply.)

23: `int one = Math.min(5, 3);`

24: `long two = Math.round(5.5);`

25: `double three = Math.floor(6.6);`

26: `var doubles = new double[] {one, two, three};`

A. 0

B. 1

C. 2

D. 3

E. 4

Mi justificación
Todas tienen notación válida. La línea 23 compara dos enteros, así que <code>int one</code> no presenta errores. La línea 24 está haciendo <code>long = int</code> , lo cual es compatible sin cast explícito. La línea 25 es válida ya que <code>double = int</code> es compatible sin cast explícito. La línea 26 es válida ya que <code>var</code> puede usarse para arreglos.

10. A. Todas estas líneas compilan. Los métodos `min()` y `floor()` devuelven el mismo tipo que reciben: `int` y `double`, respectivamente. El método `round()` devuelve un `long` cuando recibe un `double`. La opción A es correcta porque el código compila.

11. What is the output of the following code?

```
var date = LocalDate.of(2022, 4, 3);  
date.plusDays(2);  
date.plusHours(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
    + " " + date.getDayOfMonth());
```

- A. 2022 MARCH 4
- B. 2022 MARCH 6
- C. 2022 APRIL 3
- D. 2022 APRIL 5
- E. The code does not compile.
- F. A runtime exception is thrown.

Mi justificación
La variable date es un LocalDate, al cual no le puedo sumar horas ya que no las incluye en su formato y por tanto no tiene dicho método implementado, por lo tanto el código no compila

11. E. Un LocalDate no tiene un elemento de tiempo. Por lo tanto, no existe un método para agregar horas, haciendo que la opción E sea la respuesta.

12. What is output by the following code? (Choose all that apply.)

```
var numbers = "012345678".indent(1);  
numbers = numbers.stripLeading();  
System.out.println(numbers.substring(1, 3));  
System.out.println(numbers.substring(7, 7));  
System.out.print(numbers.substring(7));
```

- A. 12
- B. 123
- C. 7
- D. 78
- E. A blank line
- F. The code does not compile.
- G. An exception is thrown.

Mi justificación
El substring(1,3) corresponde a los elementos con índice 1 y 2, así que imprime 12. El substring(7,7) imprime "". El substring(7) imprime del índice 7 en adelante así que imprimiría 78.

12. A, D, E. Primero, observa que la llamada a `indent()` agrega un espacio en blanco al comienzo de los números, y `stripLeading()` lo elimina inmediatamente. Por lo tanto, estos métodos se cancelan entre sí y no tienen ningún efecto. El método `substring()` tiene dos formas. La primera recibe el índice donde comenzar y el índice donde detenerse inmediatamente antes. La segunda recibe solamente el índice donde comenzar y continúa hasta el final del String. Recuerda que los índices comienzan en cero. La primera llamada comienza en el índice 1 y termina en el índice 2, ya que debe detenerse antes del índice 3. Esto da como resultado la opción A. La segunda llamada comienza en el índice 7 y termina en el mismo lugar, dando como resultado un String vacío, que corresponde a la opción E. Esto imprime una línea en blanco. La última llamada comienza en el índice 7 y continúa hasta el final del String, terminando con la opción D.

13. What is the result of the following code?

```
public class Lion {  
    public void roar(String roar1, StringBuilder roar2) {  
        roar1.concat("!!!");  
        roar2.append("!!!");  
    }  
    public static void main(String[] args) {  
        var roar1 = "roar";  
        var roar2 = new StringBuilder("roar");  
        new Lion().roar(roar1, roar2);  
        System.out.println(roar1 + " " + roar2);  
    }  
}
```

- A. roar roar
- B. roar roar!!!
- C. roar!!! roar
- D. roar!!! roar!!!
- E. An exception is thrown.
- F. The code does not compile.

Mi justificación
El código no compila porque roar1.concat("!!!") debería ser asignado a otro String para que funcione, no puedes simplemente esperar que se concatene al mismo ya que los String son inmutables

13. B. Un String es inmutable. Llamar a concat() devuelve un nuevo String, pero no modifica el original. Un StringBuilder es mutable. Llamar a append() agrega caracteres a la secuencia de caracteres existente y, además, devuelve una referencia al mismo objeto. Por lo tanto, la opción B es correcta.

14. Given the following, which can correctly fill in the blank? (Choose all that apply.)

```
var date = LocalDate.now();  
var time = LocalTime.now();  
var dateTime = LocalDateTime.now();  
var zoneId = ZoneId.systemDefault();  
var zonedDateTime = ZonedDateTime.of(dateTime, zoneId);  
Instant instant = _____;
```

- A. `Instant.now()`
- B. `new Instant()`
- C. `date.toInstant()`
- D. `dateTime.toInstant()`
- E. `time.toInstant()`
- F. `zonedDateTime.toInstant()`

Mi justificación
El método toInstant solo funciona para ZonedDateTime y OffsetDateTime, así que C,D y E son inválidas. Las demás no tienen problema.

14. A, F. La opción A crea correctamente el instante actual. La opción F también es una conversión válida. La opción B es incorrecta porque Instant, al igual que muchas otras clases de fecha/hora, no tiene un constructor público y se instancia mediante métodos. Las opciones C, D y E son incorrectas porque el objeto de origen no representa un punto específico en el tiempo. Sin una zona horaria, Java no sabe qué momento utilizar para el Instant.

15. What is the output of the following? (Choose all that apply.)

```
var arr = new String[] { "PIG", "pig", "123"};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
System.out.println(Arrays.binarySearch(arr, "Pippa")); }
```

- A. [pig, PIG, 123]
- B. [PIG, pig, 123]
- C. [123, PIG, pig]
- D. [123, pig, PIG]
- E. -3
- F. -2
- G. The results of `binarySearch()` are undefined in this example.

Mi justificación
Arrays.sort ordena los elementos en orden alfabético, y quedaría [123, PIG, pig] ya que de forma sencilla el orden alfabético sigue una secuencia números -> inicia con mayúscula -> inicia con minúscula. El método <code>binarySearch</code> no encuentra Pippa así que da -1, y se le resta el número del índice que ocuparía de existir, el cual sería 2 ya que estaría entre PIG y pig. Así que $-1-2 = -3$.

15. C, E. Los números se ordenan antes que las letras y las letras mayúsculas antes que las minúsculas. Esto hace que la opción C sea una de las respuestas. El método `binarySearch()` analiza dónde se insertaría un valor. Para Pippa, se insertaría antes del segundo elemento. Después, niega esa posición y le resta uno, lo cual corresponde a la opción E.

16. What is included in the output of the following code? (Choose all that apply.)

```
var base = "ewe\nsheep\\t";  
int length = base.length();  
int indent = base.indent(2).length();  
int translate = base.translateEscapes().length();  
  
var formatted = "%s %s %s".formatted(length, indent, translate);  
System.out.format(formatted);
```

- A. 10
- B. 11
- C. 12
- D. 13
- E. 14
- F. 15
- G. 16

Mi justificación

Contando todos los caracteres obtenemos que base tiene 11 caracteres (\n y \\ cuentan como un solo carácter cada uno). Así que length = 11.
Si hacemos indent(2) entonces se le agrega 2 espacios al inicio y 2 espacios al final de la frase, entonces nos queda 11 + 2 + 2 = 15, así que indent = 15.
El método translate deshace el indent, así que volvemos a tener 11, por lo tanto translate = 11.
Al final de valores únicos se imprimen 11 y 15.

16. A, B, G. Hay 11 caracteres en base porque existen dos caracteres de escape. El \n cuenta como un carácter que representa una nueva línea, y \\ cuenta como un carácter que representa una barra invertida. Esto hace que la opción B sea una de las respuestas. El método indent() agrega dos caracteres al principio de cada una de las dos líneas de base. Esto proporciona cuatro caracteres adicionales. Sin embargo, el método también normaliza agregando una nueva línea al final si no existe. Ese carácter adicional significa que agregamos cinco caracteres a los 11 existentes, dando como resultado la opción G. Finalmente, el método translateEscapes() convierte cualquier carácter de escape escrito como texto en su carácter de escape real, haciendo que \\t se convierta en \t. Esto elimina un carácter, dejando 10 caracteres, lo que coincide con la opción A.

17. Which of these statements are true? (Choose all that apply.)

```
var letters = new StringBuilder("abcdefg");
```

- A. `letters.substring(1, 2)` returns a single-character `String`.
- B. `letters.substring(2, 2)` returns a single-character `String`.
- C. `letters.substring(6, 5)` returns a single-character `String`.
- D. `letters.substring(6, 6)` returns a single-character `String`.
- E. `letters.substring(1, 2)` throws an exception.
- F. `letters.substring(2, 2)` throws an exception.
- G. `letters.substring(6, 5)` throws an exception.
- H. `letters.substring(6, 6)` throws an exception.

Mi justificación
✓ A es true, regresa solo el carácter en el índice 1. ✗ B es false, regresa "" ✗ C es false, da error porque el primer parámetro no debe ser mayor al segundo. ✗ D es false, regresa "" ✗ E es false ya que no tiene error ✗ F es false ya que no tiene error ✓ G es true porque el primer parámetro no debe ser mayor al segundo, así que es un error. ✗ H es false ya que regresa ""

17. **A, G.** El método `substring()` incluye el índice inicial, pero no el índice final. Cuando se llama con 1 y 2, devuelve un `String` de un solo carácter, haciendo que la opción A sea correcta y la opción E incorrecta. Llamar a `substring()` con 2 como ambos parámetros es válido. Devuelve un `String` vacío, haciendo que las opciones B y F sean incorrectas. Java no permite especificar los índices en orden inverso. La opción G es correcta porque lanza una `StringIndexOutOfBoundsException`. Finalmente, la opción H es incorrecta porque devuelve un `String` vacío.

18. What is the result of the following code? (Choose all that apply.)

```
13: String s1 = ""
14:     purr"";

15: String s2 = "";
16:
17: s1.toUpperCase();
18: s1.trim();
19: s1.substring(1, 3);
20: s1 += "two";
21:
22: s2 += 2;
23: s2 += 'c';
24: s2 += false;
25:
26: if ( s2 == "2cfalse") System.out.println("==");
27: if ( s2.equals("2cfalse")) System.out.println("equals");
28: System.out.println(s1.length());
```

- A. 2
- B. 4
- C. 7
- D. 10
- E. ==
- F. equals
- G. An exception is thrown.
- H. The code does not compile.

Mi justificación
El string s1 es inmutable por ser String, y como s1.substring no se está guardando en una variable, entonces no es válido.

18. C, F. Esta pregunta es complicada porque tiene varias partes. Primero, debes saber que el bloque de texto de las líneas 13 y 14 es equivalente a un String normal. Como no hay un salto de línea al final, tiene cuatro caracteres. Después, debes saber que los objetos String son inmutables, lo que significa que los resultados de las líneas 17–19 se ignoran. Finalmente, en la línea 20 ocurre algo. Concatenamos tres caracteres nuevos a s1 y ahora tenemos un String de longitud 7, haciendo que la opción C sea correcta.

Después, s2 += 2 se expande como s2 = s2 + 2. Un String concatenado con cualquier otro tipo produce un String. Las líneas 22, 23 y 24 agregan elementos a s2, dando como resultado "2cfalse". La instrucción if de la línea 27 devuelve true porque los

valores de los dos objetos String son iguales mediante equals(). La instrucción if de la línea 26 devuelve false porque los dos objetos String no son el mismo objeto en memoria. Uno proviene directamente del *string pool* y el otro se construye mediante operaciones con String.

19. Which of the following fill in the blank to print a positive integer? (Choose all that apply.)

```
String[] s1 = { "Camel", "Peacock", "Llama"};  
String[] s2 = { "Camel", "Llama", "Peacock"};  
String[] s3 = { "Camel"};  
String[] s4 = { "Camel", null};  
System.out.println(Arrays._____);
```

- A. `compare(s1, s2)`
- B. `mismatch(s1, s2)`
- C. `compare(s3, s4)`
- D. `mismatch (s3, s4)`
- E. `compare(s4, s4)`
- F. `mismatch (s4, s4)`

Mi justificación
✓ A es válida ya que el primer elemento es igual así que va al segundo, donde s1 es mayor por orden alfabético, así que s1 daría positivo ✓ B es válida ya que s1 y s2 no son iguales, y mismatch da positivo en estos casos. ✗ C es inválida porque s3 tiene 1 elemento y s4 tiene 2, así que comparando gana s4 y eso sería un resultado con valor negativo. ✓ D es válida ya que s3 y s4 no son iguales, y mismatch da positivo en estos casos. ✗ E es inválida ya que compare da 0 ya que son iguales y 0 no es positivo ni negativo. ✗ F es inválida ya que mismatch da -1 ya que son iguales.

19. **A, B, D.** El método `compare()` devuelve un entero positivo cuando los arreglos son diferentes y el primero es mayor. Este es el caso de la opción A porque el elemento en el índice 1 aparece primero alfabéticamente. No es el caso de la opción C porque s4 es más largo, ni de la opción E porque los arreglos son iguales.

El método `mismatch()` devuelve un entero positivo cuando los arreglos son diferentes en una posición cuyo índice es 1 o mayor. Este es el caso de las opciones B y D porque la diferencia está en el índice 1. No es el caso de la opción F porque no existe ninguna diferencia.

Resultado final: 10/22

Calificación = 4.5